

PLANNING • TOOL USE • MEMORY • MULTI-AGENT • EVALUATION • GUARDRAILS • AGENTOPS

Cracking Agentic AI System Design Interviews

**From a Single Bounded Agent Loop to Multi-Agent Production Systems
with LangGraph, MCP, A2A, OpenTelemetry, and Human Oversight**

AI ENGINEERING INSIDER • aiengineeringinsider.com

65 Agentic AI System Design Prep. • 45 SENIOR INTERVIEW QUESTIONS • RUNNABLE LABS



AI ENGINEERING CAREER SERIES

Cracking Agentic AI System Design Interviews

From a Single Bounded Agent Loop to Multi-Agent Production Systems
with LangGraph, MCP, A2A, OpenTelemetry, and Human Oversight

AI Engineering Insider

aiengineeringinsider.com • aiengineeringinsider.substack.com

Companion code: github.com/lamhotsiagian/agent-ai-lab

Cracking Agentic AI System Design Interviews

First edition. Published by AI Engineering Insider.

Every architecture in this book is a teaching model. Production systems carry constraints that no book can anticipate, so treat each design as a starting point that you adapt to your own traffic, data, and regulatory position rather than as a specification to copy.

Model names, framework versions, benchmark scores, and pricing move faster than print. The book therefore emphasises the reasoning that survives a version bump and marks time-sensitive figures explicitly where they appear.

Company incidents described in case study boxes are composites drawn from publicly documented failures and from patterns the author has seen repeatedly. They are anonymised deliberately.

All source listings are released under the MIT License through the companion repository at github.com/lamhotsiagian/agentic-ai-lab.

Preface

Subscribe → aiengineeringinsider.substack.com/subscribe

Most engineers meet agentic AI through a demo. A model receives a goal, calls a few tools, and produces something that looks like reasoning. The demo works, the room applauds, and the engineer walks away with a dangerous conclusion, namely that the hard part is the prompt.

The hard part is never the prompt. The hard part is that you have introduced a nondeterministic controller into a distributed system, given it credentials, and asked it to make sequential decisions where each decision changes the state that the next decision depends on. Everything that makes distributed systems difficult now applies, and it applies to a controller that cannot be unit tested in the ordinary sense.

This book treats that situation as an engineering problem rather than as a prompting problem. It builds up from the single bounded agent loop, through tool interfaces, memory hierarchies, and multi-agent coordination, into evaluation, observability, cost control, and security. It ends with a capstone system that assembles every idea into something you can run.

Why This Book Exists

Three gaps motivated it.

The first gap is between framework tutorials and production reality. Tutorials show you how to wire a graph. They do not show you what happens when the retrieval service degrades, when a tool returns a schema the model has never seen, or when a single user submits a request that costs forty dollars in tokens. Production is mostly those cases.

The second gap is between research literature and design practice. The papers behind ReAct, Reflexion, Toolformer, and Tree of Thoughts are excellent, and none of them tell you how to bound a loop, budget a context window, or write a service level objective for a workflow with a variable number of steps.

The third gap is the interview itself. Senior interview loops for agentic roles do not test whether you have memorised the ReAct prompt template. They test whether you can state a requirement as a number, choose an architecture that meets it, name the trade-off you accepted, and describe the measurement that would tell you the choice was wrong. That skill is teachable, and this book teaches it explicitly.

Who This Book Is For

The book assumes you write production software and have used a language model API. It does not assume you have shipped an agent.

- **AI engineers and machine learning engineers** moving from model serving into agent orchestration will find the architecture, evaluation, and cost chapters most useful.

- **Software engineers and backend engineers** entering the field will recognise most of the distributed systems reasoning and can focus on Parts I, II, and IV.
- **AI architects and technical leads** designing platforms will get the most value from Parts III, VII, and IX, which cover coordination, pattern selection, and the capstone architecture.
- **Candidates preparing for interviews** should read Part VIII early to calibrate, then work backwards into the technical parts, then return.

The Companion Repository

Runnable code lives at github.com/lamhotsiagian/agent-ai-lab. Modules are organised by chapter, and each one ships a small user interface so you can exercise the behaviour rather than only read about it. Where a chapter prints a class, that class exists in the repository under the path named in the lab box.

Conventions Used in This Book

- **Defined terms** appear in bold blue on first use and are collected in the glossary and the index.
- `inline code` marks identifiers, commands, and literal values.
- `module/path.py` marks a file in the companion repository.
- **p95 latency** marks a measured quantity or a service level target.
- Numbers attached to a specific model, price, or benchmark are marked with the period they describe, because they will move.

A Note on Reading Order

Read Parts I and II in order. They establish the reference architecture, the bounded control loop, the tool contract, and the memory hierarchy that every later chapter assumes. After that, the parts are largely independent. If you have an interview next week, read Part VIII first, then Part VII, then fill gaps backwards.

Build the capstone. A candidate who has operated their own agent system answers differently, and interviewers hear the difference immediately.

Contents

Preface	iii
List of Figures	xviii
List of Tables	xx
Part I Foundations and Architecture	1
1 Introduction to Agentic AI	2
1.1 Overview: The Shift From Answering to Acting	2
1.2 Defining an Agent Precisely	3
1.3 The Architecture Gap	5
1.4 The Control Loop	6
1.5 A Working Taxonomy of Agents	11
1.6 When Not to Build an Agent	12
1.7 The Cost Model You Will Be Asked to Defend	13
1.8 Where Agentic Systems Deliver in 2026	14
1.9 Chapter Summary	14
2 Designing Agent Systems	16
2.1 Overview: Design Precedes Framework	16
2.2 The Seven-Step Design Method	16
2.3 Component Contracts	21
2.4 Choosing the Control Pattern	22
2.5 Designing the Failure Ladder	24
2.6 Testing an Agent Design Before You Build It	26
2.7 Chapter Summary	27
3 Reasoning Models and Test-Time Compute	29
3.1 Overview: Compute Moved From Training to Inference	29
3.2 How Reasoning Models Differ Mechanically	30
3.3 Routing Reasoning Budget	32
3.4 The Four Reasoning Topologies	35
3.5 Chapter Summary	39
4 Agentic Frameworks Deep Dive	41
4.1 Overview: Frameworks Are Converging	41
4.2 LangGraph: State Machines With Durability	42
4.3 The Alternatives, Compared Operationally	46
4.4 State Snapshots Versus Event-Sourced Workflows (Temporal and Cadence)	47
4.5 A Defensible Selection Rule	49
4.6 Keeping Business Logic Portable	50

4.7	Chapter Summary	52
Part I Interview Drill: Foundations and Architecture		54
PI.1	Design an agentic triage system for a B2B support desk handling 60,000 tickets per day	54
PI.2	Your agent's cost per run tripled after a prompt change that added one sentence. Explain the mechanism and the controls you would add	55
PI.3	An agent returns confident answers built from one third of the evidence it paid to retrieve. Find the bug	56
PI.4	Explain test-time compute, and tell me exactly where you would spend reasoning budget in a five-step agent	57
PI.5	Moffatt v. Air Canada: what does a tribunal ruling teach you about agent architecture?	58
Part II Core Agent Capabilities		60
5	Tool Use and the Agent-Computer Interface	61
5.1	Overview: The Interface Is the Product	61
5.2	Eight Rules of Tool Schema Design	62
5.3	Typed Results: The Five States That Matter	63
5.4	The Tool Gateway	64
5.5	The Model Context Protocol	68
5.6	Tool Selection at Scale	70
5.7	Sandboxing Code Execution	71
5.8	Chapter Summary	73
6	Orchestration and Context Engineering	74
6.1	Overview: Control Is a Resource You Allocate	74
6.2	Six Orchestration Topologies	74
6.3	Context Engineering	77
6.4	Chapter Summary	83
7	Knowledge, Memory, and Retrieval	84
7.1	Overview: Memory Is a Write Problem	84
7.2	The Four Memory Tiers	84
7.3	The Write Policy	85
7.4	The Retrieval Contract	88
7.5	Consolidation, Decay, and Forgetting	90
7.6	Multi-Tenancy, Erasure, and Audit	91
7.7	Hierarchical Memory Compaction and Lifecycle Garbage Collection	91
7.8	Bitemporal Memory: Valid-Time Versus System-Time	93
7.9	Chapter Summary	95
8	Learning in Agentic Systems	97
8.1	Overview: Four Learning Surfaces	97
8.2	Learning Without Parameter Updates	98
8.3	When Parameter Updates Earn Their Cost	103
8.4	The Data Flywheel	104
8.5	Chapter Summary	106

Part II Interview Drill: Core Agent Capabilities	107
P11.1 Design the tool and memory layer for an agent that operates across 200 internal APIs	107
P11.2 Your agent’s answer quality degrades on long runs but every component tests clean. What do you investigate?	108
P11.3 Your agent gives a customer an answer based on a preference the customer retracted three months ago. Trace the defect	109
P11.4 Explain why you would fuse retrieval results by rank rather than by score, and where else this reasoning applies	110
P11.5 The Replit agent that deleted a production database: which controls were missing?	111
 Part III Multi-Agent Systems and Protocols	 113
9 From One Agent to Many	114
9.1 Overview: The Multiplication Is Not Symmetric	114
9.2 Four Coordination Topologies	114
9.3 The Handoff Contract	116
9.4 Concurrency, State, and Partial Failure	118
9.5 Multi-Agent Sagas and Distributed Compensation	121
9.6 Distributed Deadlocks and Wait-For-Graph (WFG) Detection	123
9.7 Observability Across Agents	124
9.8 Chapter Summary	126
10 Agent Communication Protocols	127
10.1 Overview: Two Different Problems	127
10.2 The Model Context Protocol in Depth	127
10.3 The Agent-to-Agent Protocol	130
10.4 The Trust Boundary	134
10.5 Versioning and Compatibility	135
10.6 Chapter Summary	136
11 User Experience Design for Agentic Systems	138
11.1 Overview: Three Properties That Break Conventional Interfaces	138
11.2 The Autonomy Slider	139
11.3 A State Model the User Can Follow	139
11.4 Steering and Interruption	142
11.5 Approval Interfaces That Produce Real Review	143
11.6 Failing Gracefully and Preserving Partial Value	143
11.7 Reversibility as an Interface Affordance	144
11.8 Chapter Summary	146
Part III Interview Drill: Multi-Agent Systems	147
P111.1 Design a cross-organisation agent network where a client agent delegates to vendor agents it does not control	147
P111.2 Your five-agent system is nine times more expensive than the single-agent prototype. Find the money	148
P111.3 Two agents in your system delegate to each other until the run times out, and neither agent’s step limit ever fires	149

PIII.4	When would you use MCP, when A2A, and where exactly is the trust boundary?	150
PIII.5	The Chevrolet dealership chatbot that agreed to sell a car for one dollar	151
Part IV	Production Engineering	153
12	Validation and Measurement	154
12.1	Overview: A Single Accuracy Number Hides Everything	154
12.2	Component Metrics	155
12.3	Trajectory Evaluation	155
12.4	Language Model Judges, Properly Validated	158
12.5	Golden Datasets	160
12.6	Chapter Summary	163
13	Monitoring and Observability in Production	165
13.1	Overview: Three Properties That Break Ordinary Monitoring	165
13.2	The Span Hierarchy	166
13.3	Redaction and the Two-Store Model	170
13.4	Sampling That Keeps the Runs You Need	170
13.5	Service Level Indicators for Nondeterministic Systems	171
13.6	Alerting on Causes Rather Than Symptoms	171
13.7	A Debugging Procedure That Starts From the Trace	172
13.8	Chapter Summary	174
14	Cost Optimization and FinOps for Agents	176
14.1	Overview: The Term That Surprises Everyone	176
14.2	Lever One: Prompt Prefix Caching	177
14.3	Lever Two: Model Tier Routing	178
14.4	Lever Three: Observation Projection and Truncation	178
14.5	Lever Four: Early Termination	178
14.6	Lever Five: Semantic Answer Caching	179
14.7	Lever Six: Batching and Asynchrony	181
14.8	Lever Seven: Self-Hosting, and the Break-Even Point	181
14.9	The Cost Governor	182
14.10	Chapter Summary	184
15	Improvement Loops	186
15.1	Overview: Improvement Is a Pipeline, Not a Project	186
15.2	Feedback Signals and Their Biases	186
15.3	Triage: From Traces to Ranked Failure Classes	188
15.4	Routing a Fix to the Cheapest Surface	190
15.5	Every Incident Becomes a Test	190
15.6	Progressive Rollout and Automatic Rollback	192
15.7	Experiment Design for Agents	193
15.8	Chapter Summary	194
Part IV	Interview Drill: Production Engineering	196
PIV.1	Design the complete measurement and operations stack for an agent going to production next quarter	196

PIV.2 Availability is 100 percent, latency is healthy, error rate is zero, and customers say the agent has become useless. Where do you look?	197
PIV.3 An offline evaluation says the new version is four points better. Production says it is two points worse. Reconcile them	198
PIV.4 Explain prompt prefix caching, why it fails silently, and how you would enforce it	199
PIV.5 Zillow Offers: what does an automated-decision failure teach an agent engineer?	200
Part V Safety, Security, and Governance	202
16 Protecting Agentic Systems	203
16.1 Overview: What Actually Changed	203
16.2 The Threat Taxonomy	204
16.3 Architectural Controls	205
16.4 The Defence Stack in Order	208
16.5 Detection	210
16.6 Threat Modelling in a Design Review	211
16.7 Chapter Summary	212
17 Responsible AI and Ethics for Agents	214
17.1 Overview: Acting Raises the Stakes	214
17.2 Fairness for Systems That Act	215
17.3 The Decision Record	217
17.4 The Regulatory Landscape	219
17.5 Chapter Summary	223
18 Human-Agent Collaboration and Governance	224
18.1 Overview: Three Roles, Not One	224
18.2 Gate Placement and the Capacity Constraint	225
18.3 The Escalation Router	226
18.4 The Authority Model	228
18.5 Automation Bias and Its Counters	230
18.6 Incident Response for Agents	230
18.7 Chapter Summary	233
Part V Interview Drill: Safety, Security, and Governance	234
PV.1 Design the security architecture for a support agent that reads customer records and browses the public web	234
PV.2 Your guardrail block rate fell from 0.4 percent to zero overnight and nobody deployed anything. What do you do?	235
PV.3 A customer reports that their account data appeared on a third-party website. Your agent never called an external API. Explain how	236
PV.4 Explain prompt injection to a security architect who is sceptical that it is different from SQL injection	237
PV.5 The Cursor support bot that invented a policy: what does a hallucinated rule cost, and how do you prevent it?	238

Part VI Frontier Capabilities	240
19 Computer Use and Multimodal Agents	241
19.1 Overview: When the Screen Is the Only Interface	241
19.2 The Reliability Arithmetic	241
19.3 The Grounding Problem	243
19.4 Cost and Latency	246
19.5 The Production Perception Ladder and Vision Token Economics	246
19.6 What the Benchmarks Measure	248
19.7 Containment	249
19.8 Chapter Summary	251
20 Edge, On-Device, and Embedded Agents	252
20.1 Overview: Four Reasons, Only One of Which Is Usually Real	252
20.2 The Arithmetic That Predicts Performance	253
20.3 The Capability Envelope of Small Models	254
20.4 Three Hybrid Topologies	254
20.5 Model Lifecycle on Hardware You Do Not Control	257
20.6 Chapter Summary	259
21 Autonomous Code Generation	260
21.1 Overview: Why This One Worked First	260
21.2 What the Benchmarks Measure and Omit	260
21.3 Architecture	261
21.4 The Verification Ladder	264
21.5 Reward Hacking	264
21.6 Review, Blast Radius, and Merge Discipline	267
21.7 Chapter Summary	269
22 Agentic RAG and Dynamic Knowledge Graphs	270
22.1 Overview: Retrieval Is a Decision, Not a Stage	270
22.2 Question One: Should I Retrieve?	271
22.3 Question Two: From Where, and With What Query?	271
22.4 Question Three: Is What I Got Good Enough?	273
22.5 Fusing Heterogeneous Evidence	276
22.6 Chapter Summary	279
Part VI Interview Drill: Frontier Capabilities	280
PVI.1 Design an agent that processes 2,000 insurance claims per day through a legacy desktop application with no API	280
PVI.2 Your on-device assistant is fast on the test bench and unusably slow in the field. Diagnose it	281
PVI.3 Your coding agent's pull requests pass every check and a third of them get reverted within a month. Find the gap	282
PVI.4 Walk me through corrective retrieval, and tell me what you would build first if you only had a week	283
PVI.5 The McDonald's AI drive-through pilot: what does a real-time multi-modal deployment teach you?	284

Part VII System Design Deep Dives	286
23 System Design Patterns for Agents	287
23.1 Overview: Why Names Matter	287
23.2 Family One: Orchestration Patterns	287
23.3 Family Two: Context and Memory Patterns	288
23.4 Family Three: Tool and Action Patterns	289
23.5 Family Four: Reliability Patterns	290
23.6 Family Five: Safety Patterns	290
23.7 Family Six: Cost and Evaluation Patterns	291
23.8 Six Patterns in Depth	292
23.9 Composition Rules	295
23.10 Chapter Summary	298
24 Designing for Scale and Resilience	299
24.1 Overview: Why Ordinary Autoscaling Misleads	299
24.2 Capacity Arithmetic	300
24.3 Admission Control and Backpressure	300
24.4 Exactly-Once Effects Under Retries and Restarts	303
24.5 Multi-Region and Multi-Provider Resilience	304
24.6 Service Level Objectives Under Nondeterminism	305
24.7 Chapter Summary	307
25 Case Studies: Production Agentic Systems	308
25.1 How to Read These	308
25.2 Case Study One: Enterprise Research Assistant	308
25.3 Case Study Two: Customer Support Agent at Scale	310
25.4 Case Study Three: Claims Processing on a Legacy Application	312
25.5 What the Three Have in Common	314
25.6 Chapter Summary	315
Part VII Interview Drill: System Design Deep Dives	317
PVII.1 Design an agent platform that ten internal product teams build on, at 50,000 runs per day	317
PVII.2 Your p95 latency doubled with no code change and no traffic increase. Work the problem	318
PVII.3 Under load your agent occasionally performs the same irreversible action twice. Nothing in the code retries. Explain	319
PVII.4 Size the worker pool for an agent service, and tell me which signal you would autoscale on	320
PVII.5 Knight Capital, 2012: what does a 45-minute automated trading failure teach an agent platform team?	321
Part VIII Interview Preparation	323
26 The Agentic AI Interview Roadmap	324
26.1 Overview: Titles Are Noise, Loops Are Signal	324
26.2 Six Role Archetypes	324
26.3 The Stages and What Each One Scores	326

26.4	Self-Assessment Before You Plan	326
26.5	The Ninety-Day Plan	327
26.6	Portfolio Projects That Produce Signal	327
26.7	Positioning Without Agent Experience	329
26.8	Why Candidates Get Rejected	330
26.9	Chapter Summary	331
27	Technical Interview: Concepts and Coding	332
27.1	How the Technical Round Is Scored	332
27.2	Forty Concept Answers at Interview Length	333
27.3	The Eight Recurring Coding Problems	336
27.4	What Strong Live Coding Sounds Like	341
27.5	Recovering When It Goes Wrong	342
27.6	Chapter Summary	342
28	System Design Interview: Agentic Systems	344
28.1	The Forty-Five Minute Script	344
28.2	Worked Design One: A Deep Research Agent at Consumer Scale	346
28.3	Worked Design Two: An Incident Response Agent for an SRE Team	348
28.4	Follow-Up Questions and the Shape of a Strong Answer	349
28.5	The Levelling Rubric	349
28.6	Recovering a Round That Has Gone Sideways	351
28.7	Chapter Summary	352
29	Behavioural and Craft Questions	353
29.1	Why This Round Carries More Weight Here	353
29.2	Story Structure That Forces Specifics	354
29.3	The Twelve Stories	354
29.4	Responsible AI Judgement Questions	356
29.5	Ambiguity and Influence	357
29.6	Telling a Failure Story That Raises Your Level	358
29.7	Questions Worth Asking	359
29.8	Levelling and Negotiation	359
29.9	Chapter Summary	360
Part VIII	Interview Drill: Interview Preparation	362
PVIII.1	You have 45 minutes. Design an AI agent that triages and drafts responses to inbound security questionnaires. Go	362
PVIII.2	You are given a four-hour take-home: build an agent that answers questions over a document set. What do you build?	363
PVIII.3	In a live round the interviewer shows you a trace and says “this run cost \$4.10 and should have cost 20 cents. Find it.” How do you work it?	364
PVIII.4	The interviewer says “explain how you would evaluate an agent” and then stays silent for two minutes. What do you cover, and in what order?	365
PVIII.5	DoNotPay and the FTC: what does an enforcement action over AI capability claims teach an engineer?	366

Part IX Capstone: AgentOps Studio	368
30 AgentOps Studio: End-to-End Architecture	369
30.1 What You Are Building and Why	369
30.2 Requirements	369
30.3 The Module Map	370
30.4 Architecture	371
30.5 The Typed State	371
30.6 The State Machine	374
30.7 Tool Registry and Authorisation	375
30.8 The Streaming API Surface	377
30.9 Build Order	377
30.10 Chapter Summary	379
31 Evaluation, Ingestion, and Training Maturity	381
31.1 Why Ingestion Comes Before Retrieval Tuning	381
31.2 The Ingestion Pipeline	381
31.3 Measuring Ingestion Rather Than Trusting It	385
31.4 The Evaluation Harness, Concretely	386
31.5 The Training Maturity Model	388
31.6 Chapter Summary	391
32 Deployment, Operations, and Lessons Learned	392
32.1 Deployment Topology	392
32.2 Identity and Authorisation	394
32.3 The Audit Log	395
32.4 Production Hardening Checklist	396
32.5 Runbooks for the Four Incident Classes	398
32.6 Migrations: Prompts, Schemas, and Corpora	398
32.7 Lessons the Design Only Teaches After You Operate It	398
32.8 Chapter Summary	400
Part IX Interview Drill: The Capstone	402
PIX.1 Extend AgentOps Studio to serve three tenant tiers with different guarantees. Design it	402
PIX.2 You operated the capstone for three weeks. What broke, and what does that tell me about you?	403
PIX.3 A capstone run returns an answer citing a document the user deleted last week. Trace it	404
PIX.4 Defend one design decision in your capstone that a reviewer pushed back on	405
PIX.5 Slack AI and indirect prompt injection: what would your capstone have done differently?	406
Part X Bonus: Mock Interview Masterclass	408
33 AI Agent System Design Mock Interviews	409
33.1 Foundation Models	410
33.2 Agent Frameworks	414

33.3	Memory Systems	418
33.4	Vector Databases & Knowledge Stores	422
33.5	Multi-Agent Coordination	426
33.6	Tool & Action Layer	430
33.7	Data Ingestion & Environment Perception	434
33.8	Planning & Reasoning Layer	438
33.9	Embeddings & Representation	442
33.10	Execution & Runtime	446
33.11	Evaluation, Safety & Observability	450
33.12	Guardrails & Governance	454
33.13	Quick Reference: Key Themes Across Topics	458
 Part XI Reference Material		459
Appendix A: Chapter Summary Table		460
Appendix B: Glossary		463
Appendix C: References		469
Appendix D: About the Author and Next Steps		473
Index		474

List of Figures

1.1	The six-component agent reference architecture. The orchestrator owns control flow, the model proposes, and the gateway is the only path to side effects.	6
1.2	One iteration of the bounded control loop, drawn as a sequence over time. Every arrow that leaves the orchestrator is budget checked before it is sent.	7
2.1	Layered agent architecture. Dependencies point downward only, so any layer can be tested with the layer below it replaced by a fake.	21
2.2	Plan, execute, verify, and bounded replan. The replan counter is separate from the step counter, which is what prevents a plausible-looking infinite cycle.	23
3.1	Accuracy against reasoning budget for three task classes. The saturation point differs by class, which is why a single global effort setting wastes money on easy traffic and starves hard traffic.	31
3.2	A three-stage reasoning router. Deterministic signals resolve most traffic, a small classifier resolves the remainder, and escalation on failed verification catches the rest.	32
3.3	ReAct against ReWOO for a three-tool task. ReAct pays a model call per step and serialises; ReWOO pays two model calls and parallelises, at the cost of planning without evidence.	36
4.1	A production LangGraph topology for a research agent. Solid edges are static, dashed edges are computed at runtime, and the red edge is the bounded repair path.	42
4.2	Interrupt and resume across a process boundary. The run survives because state is durable, and the approval decision arrives as an ordinary resume command carrying the thread identifier.	46
5.1	Tool invocation with the checks in the correct order. Authorisation runs before validation so that a denied tool never reveals its schema through error messages.	65
5.2	Model Context Protocol topology. The host application owns one client per server, and each server exposes tools, resources, and prompts over a transport the host chooses.	68
6.1	Six orchestration topologies. Solid boxes are deterministic control, violet boxes are model-controlled decisions, and the shaded region marks where cost variance enters.	75
6.2	Deterministic context assembly. Sections have fixed order and fixed budgets, and eviction happens inside a section rather than across the whole window.	78
6.3	Sub-agent isolation. The parent's window grows by one summary per subtask instead of by every observation the subtask consumed.	81
7.1	The four-tier memory hierarchy. Latency and volatility fall as you move down; cost per read and durability rise.	85

7.2	One memory read cycle. Tenancy is enforced in the store, the fusion stage combines lexical and semantic candidates, and the caller receives a hard token bound.	88
8.1	The skill promotion pipeline. Nothing reaches the production library without passing an offline evaluation and a shadow period, and every skill can be demoted automatically.	100
8.2	The data flywheel with its three safety gates. Deduplication, leakage prevention, and human adjudication each block a specific way that trace-derived training data goes wrong.	104
9.1	Four coordination topologies. Interaction paths grow from linear in the supervisor and pipeline cases to quadratic in the network case, and the failure surface grows with them.	115
9.2	A supervisor run with parallel workers, a shared budget, and a partial-failure path. The supervisor synthesises from what returned rather than failing the run when one worker is unavailable.	119
10.1	The protocol stack. MCP runs vertically from an agent to its capabilities. A2A runs horizontally between agents that each own their own tool layer.	128
10.2	An A2A task lifecycle including capability discovery, a mid-task input request, streaming progress, and artifact delivery. The peer's internal tool calls are deliberately invisible to the client.	131
11.1	The autonomy slider. Each level changes where the human sits in the loop, which changes both throughput and the cost of an error.	139
11.2	Mid-run steering. The constraint enters state and takes effect at the next step boundary, and the interface confirms it explicitly rather than silently.	143
12.1	The evaluation pyramid. Cost per case rises and volume falls as you move up, so the layers below must catch most defects before the expensive layers run.	154
12.2	An evaluation run inside continuous integration. Safety assertions gate absolutely, quality metrics gate against the baseline with a significance test, and cost is reported so a quality gain that triples spend is a visible trade.	162
13.1	Span hierarchy for a single agent run. Every span carries cost and token attributes, so cost attribution by step, tool, tenant, or code path is a query rather than an investigation.	166
13.2	The debugging path from an alert to a root cause. Each step narrows using data the instrumentation already carries, so no step requires reproducing the failure. . .	173
14.1	A cost waterfall for a representative six-step research run, showing where each optimisation lever removes spend. The ordering reflects typical measured impact. .	177
14.2	The cached prefix boundary. Everything above it must be byte-stable across every request in a class; everything below it varies freely.	178
15.1	The improvement loop. Every arrow is scheduled work with an owner, and the regression suite grows monotonically so a fixed defect stays fixed.	187
15.2	Progressive rollout with guardrail metrics evaluated at each stage. Rollback is automatic and does not wait for a human, because the human is asleep at the moment it matters.	192

16.1	The lethal trifecta. Each pair is defensible; the intersection is not. Design so that no single agent identity holds all three, and enforce the separation in the gateway rather than in the prompt.	205
16.2	Privilege separation. Untrusted content reaches only the quarantined model, whose output crosses the boundary as schema-validated values that can never be interpreted as instructions.	206
16.3	Defence in depth for a single tool call, showing which layer stops which threat. Every layer assumes the layer above it will sometimes fail.	209
17.1	Governance artefacts mapped to the lifecycle stage that produces them. Each artefact is a by-product of engineering work rather than a separate compliance exercise.	221
18.1	Escalation routing. The trigger determines the queue, the queue has its own capacity and service level, and expiry is a designed outcome rather than an accident.	226
18.2	Agent incident response. The first three steps run in parallel, because determining blast radius takes longer than stopping the bleeding and both must start immediately.	231
19.1	The perceive, ground, act, verify loop. Verification after each action is what converts a compounding failure product into a sequence of independently recoverable steps.	243
19.2	Deterministic replay with fallback. A recorded trajectory executes without model calls; a verification failure re-enters the perception loop and updates the recording.	246
20.1	Three hybrid topologies. Local-first escalates on measured confidence, cascade escalates on verification, and split planning uses the cloud once and executes locally.	255
21.1	A production coding agent. Localisation narrows the repository before any generation happens, and the verification ladder runs cheapest checks first so most failures cost milliseconds.	261
22.1	The agentic retrieval decision flow. Each diamond is a runtime decision, and each has a cheap measurable signal behind it rather than a model's opinion.	271
22.2	Corrective retrieval in sequence. The grade is computed from measurable signals, correction escalates through rewrite, retriever change, and external fallback, and the loop refuses rather than searching indefinitely.	277
23.1	How the patterns compose in a representative production system. Each labelled box is a pattern boundary, and each boundary is where a control lives.	296
24.1	Admission control under load. Requests are classified, checked against concurrency and quota, and then admitted, degraded, queued, or rejected. Every path is explicit and observable.	301
24.2	Failover behaviour under a provider incident. The breaker opens, the router shifts to the secondary provider, and the degradation ladder discloses the change rather than hiding it.	304
25.1	Research assistant architecture. Sub-agents isolate per-source research, an evidence ledger removes duplicate retrieval, and the grader gates synthesis.	310

25.2	Support agent architecture. Most traffic follows deterministic workflows; the bounded loop serves the residual tail, and writes stage to an approval queue. . . .	311
26.1	Where the archetypes sit against two axes: how much of the job is building the model layer versus the system around it, and how much is engineering versus stakeholder work.	325
28.1	The forty-five minute script. The first eight minutes buy the rest of the round, and the last five are where levelling actually happens.	344
28.2	Deep research agent. Untrusted web content is confined to sub-agents that hold no user document access, and their conclusions cross a schema boundary into the synthesiser.	347
28.3	Incident response agent. Investigation is read-only and highly autonomous; any mitigation stages to an approval gate with the evidence attached, and the correlation engine runs deterministically rather than through the model.	348
29.1	The story matrix. Most questions map to more than one story, so decide in advance which story you lead with for each question rather than choosing under pressure. .	354
30.1	AgentOps Studio, complete. Dependencies point downward, the gateway is the only path to effects, and provenance flows from ingestion through to the guard layer.	371
30.2	The AgentOps state machine. Solid edges are static, dashed edges are computed at runtime, red edges are bounded repair paths, and the amber node is the human interrupt.	375
31.1	Ingestion for AgentOps Studio. Content addressing makes deduplication and erasure exact, provenance travels with every chunk, and the corpus version is what the answer cache keys on.	382
31.2	The maturity ladder. Each rung costs roughly an order of magnitude more than the one below and should only be attempted once that one is exhausted.	389
32.1	Deployment topology. The API admits and streams; workers execute runs against durable state; the sandbox is isolated with no ambient credentials and no egress. .	392
32.2	Identity propagation. The end user's principal travels from the edge into the run and out to every backing system, so authorisation is enforced where the data lives.	394

List of Tables

1.1	What changed between a chat application and an agentic application	3
1.2	Six levels of agency, with the control point at each level	5
1.3	Practical agent taxonomy, with the operational consequence of each type	12
2.1	The eight numbers to establish before drawing a single box	17
2.2	Action space classification, which drives guardrail placement	20
2.3	Control patterns with the condition that selects each one	23
2.4	A degradation ladder for a research agent, from full capability to refusal	24
3.1	Instruction-tuned model versus reasoning model, from a system design perspective	31
3.2	Reasoning topologies, with cost, latency, and the condition that selects each	36
4.1	Framework comparison on operational dimensions	47
5.1	Schema anti-patterns and their corrected form	63
5.2	Strategies for exposing many tools without destroying selection accuracy	71
6.1	Orchestration topologies with their selection condition and dominant risk	75
6.2	Context strategies for a long-horizon run, with their cost and loss profile	81
7.1	Memory tiers with query pattern, storage fit, and dominant failure mode	85
8.1	Where learning is stored, with iteration cost and reversibility	97
8.2	Parameter update methods with the signal each requires	103
9.1	What a second agent buys and what it costs	115
10.1	MCP transports and their operational consequences	128
10.2	Protocol selection, stated as a decision procedure	134
11.1	Agent states, what the user sees, and what they can do in each	140
12.1	Component metrics, their definition, and the defect each one localises	155
13.1	Span attributes that make a run reconstructable and cost attributable	167
13.2	A service level objective set for a production agent	172
15.1	Feedback signals, their strength, and their characteristic bias	187
15.2	Failure class to surface routing, with the typical fix and its lead time	190
16.1	Agent-specific threats, their mechanism, and the layer that must stop them	204
17.1	Where disparity enters an agent trajectory, and how to detect each	215

17.2 Major frameworks and the engineering artefact each one demands. Verify current status and dates, since these have been subject to ongoing amendment.	220
18.1 Gate placement by action class, with the compensating control when a gate is removed	225
19.1 Interface options ranked, with the condition under which each is correct	242
19.2 End-to-end success against per-action accuracy and horizon length, without recovery	242
20.1 Quantisation levels and what each costs in agentic behaviour specifically	253
20.2 What small models do adequately on device and what they do not	254
21.1 The verification ladder, ordered by cost, with what each rung catches	264
22.1 Which index answers which question shape	272
23.1 Orchestration patterns	288
23.2 Context and memory patterns	289
23.3 Tool and action patterns	289
23.4 Reliability patterns	290
23.5 Safety patterns	291
23.6 Cost and evaluation patterns	292
24.1 Utilisation and its effect on queueing delay, as a planning heuristic	300
24.2 A worked objective set with error budget consequences	305
25.1 Case one requirements	309
25.2 Case two requirements	311
25.3 Case three requirements	313
25.4 Patterns common to all three systems, and where they differed	314
26.1 Role archetypes, what the work is, and what the loop weights	325
26.2 A representative loop, with the score sheet behind each stage	326
26.3 The self-assessment rubric across seven core dimensions	327
26.4 Ninety-day preparation schedule with weekly exit criteria	328
27.1 The eight problems, what they test, and the extension to expect	341
28.1 Common follow-ups and what a strong answer contains	350
28.2 What each level sounds like in a system design round	350
29.1 The twelve stories, what each one is for, and what it must contain	355
30.1 AgentOps Studio requirements, stated as the eight numbers	370
30.2 API surface. Runs are jobs with a stream, not synchronous requests.	378
31.1 Ingestion diagnostics and the downstream symptom each one prevents	386
31.2 Maturity levels with the exhaustion test that permits climbing	389
32.1 Audit event types and the field that makes each one useful	396

PART I

Foundations and Architecture

An agent is not a language model with a for loop wrapped around it. An agent is a distributed control system whose policy happens to be a probabilistic function, and almost every production incident in this field traces back to treating the second half of that sentence as if it cancelled the first.

These four chapters build the mental model the rest of the book assumes. Chapter 1 fixes the vocabulary and separates agency from autonomy, because interviewers probe that seam immediately. Chapter 2 turns the vocabulary into an architecture with named components, explicit contracts, and a state machine you can draw on a whiteboard. Chapter 3 explains where the reasoning actually comes from, and why test-time compute changed the cost model of every design in this book. Chapter 4 surveys the frameworks and gives you a defensible way to choose one under interview pressure.

Read this part in order. Every later chapter assumes the six-component reference architecture and the bounded control loop defined here.

Introduction to Agentic AI

Subscribe → aiengineeringinsider.substack.com/subscribe

What an agent actually is, what the language model underneath it cannot do alone, and the six-component reference architecture every later chapter assumes.

What This Chapter Delivers

By the end of this chapter you will define an agent precisely enough to defend the definition in an interview, place any system on a six-level agency spectrum, name the six components of the reference architecture and the contract each one satisfies, write a bounded control loop that terminates under every failure mode, and apply a decision framework that tells you when building an agent is the wrong choice. The chapter closes with the cost model that makes every later design decision quantitative.

1.1 Overview: The Shift From Answering to Acting

A language model completes text. Given a prompt, it produces a distribution over the next token, samples from it, appends the result, and repeats. The process is stateless between requests, has no side effects on the world, and terminates when a stop condition fires. That description is complete, and it explains both the capability and the ceiling.

An **agent** removes the ceiling by wrapping the model in machinery it does not have: persistent state, an interface to external systems, a policy for deciding what to do next, and a termination condition that the model does not control unilaterally. The model becomes a component inside a system rather than the system itself. Consequently, the engineering discipline shifts from prompt craft toward distributed systems design, and that shift is the subject of this book.

The distinction matters commercially. A model that answers questions sells a subscription. A system that completes tasks sells an outcome, and outcomes carry liability, latency budgets, audit requirements, and a cost per unit of work that finance will eventually ask you to defend. Every chapter that follows exists because one of those four constraints bites in production.

★ The One Sentence That Frames the Whole Book

An agent is a nondeterministic controller with credentials operating inside a distributed system. Every difficulty in this book follows from taking all three of those clauses seriously at the same time.

1.1.1 Why 2023 Through 2026 Changed the Calculus

Three independent capability shifts made agentic systems viable, and no single one of them would have been sufficient.

Reliable structured output. Early function-calling implementations produced malformed arguments often enough that engineers wrote parsers with retry ladders. Constrained decoding and schema-enforced tool calling moved argument validity from roughly nine attempts in ten to a rate where the remaining failures are semantic rather than syntactic. That change is what makes a tool call an ordinary remote procedure call rather than a parsing exercise.

Long and usable context. Context windows grew by two orders of magnitude, and, more importantly, retrieval accuracy inside those windows improved. A window you cannot use reliably in its middle is not really a window. Long context does not remove the need for memory design, as Chapter 7 argues at length, but it removes the need to compress every intermediate result aggressively.

Test-time compute. Reasoning models trained to spend additional tokens before answering shifted planning quality from a prompt-engineering problem to a budget allocation problem. Chapter 3 develops the consequences. For now, observe that you can now buy reasoning quality with money and latency, which means planning became a tunable rather than a constant.

Table 1.1: What changed between a chat application and an agentic application

Property	Chat application	Agentic application
Unit of work	One request, one response	One goal, many steps, variable count
State	Conversation transcript	Transcript, scratchpad, tool results, plan, budget ledger
Side effects	None outside the transcript	Writes to databases, sends messages, spends money
Latency profile	Roughly normal around a mean	Long tailed, unbounded without explicit bounds
Cost per request	Predictable within a factor of two	Varies by one to two orders of magnitude
Failure mode	Wrong answer	Wrong answer, or a correct answer applied to the wrong system
Testing	Input and output pairs	Trajectories, and the same input may take different valid paths
Security surface	Prompt injection alters text	Prompt injection alters actions

1.2 Defining an Agent Precisely

Interviewers open with this question because the answer reveals immediately whether a candidate has shipped anything. Vague answers mention autonomy and reasoning. Precise answers name properties that you could verify by reading code.

Agent

An agent is a software system that pursues a goal by repeatedly selecting actions from an available action space, executing them against an environment, observing the results, and updating its internal state, where the selection policy is implemented wholly or partly by a

language model and where the sequence and number of actions are not fixed in advance.

Four properties carry the definition, and each one is independently testable.

Goal-directedness. The system receives an objective rather than an instruction. “Reconcile the September invoices against the ledger and flag discrepancies above one percent” is a goal. “Summarise this text” is an instruction. The difference shows up in the code as the presence or absence of a success predicate that is evaluated at runtime.

Environment interaction. The system reads and writes state outside itself through a defined action space. Without an action space there is no agency, only text generation. In practice the action space is your tool registry, and Chapter 5 argues that its design determines most of your reliability.

Persistent state across steps. The system carries information forward: what it has tried, what it learned, how much budget remains. A stateless chain of model calls is a pipeline, not an agent, regardless of how many calls it makes.

Autonomy over control flow. The system decides which action comes next based on what it observed, rather than following a graph fixed at design time. This property is the one that separates an agent from a workflow, and it is also the property that costs you predictable latency, predictable cost, and straightforward testing.

! The Definition Trap

Candidates frequently define an agent as “an LLM that uses tools.” A fixed pipeline that calls a search API, then a summariser, then a formatter uses tools and is not an agent, because the control flow never depends on what came back. Autonomy over control flow is the discriminating property. State that explicitly and you separate yourself from most of the field in the first ninety seconds.

1.2.1 The Agency Spectrum

Agency is continuous, not binary. Placing a system on a spectrum is more useful than arguing about whether it qualifies, and interviewers respond well to candidates who reach for the spectrum instead of the label.

Most production systems in 2026 sit at levels two and three. Level four appears in research assistants and coding agents where verification is cheap. Level five remains largely experimental outside narrow domains, because the failure cost grows with the horizon while verification quality does not.

✓ Design at the Lowest Level That Meets the Requirement

Choose the lowest level of agency that satisfies your functional requirement, then justify each increment with a measurement. Every level upward buys capability and sells away predictable latency, predictable cost, testability, and auditability. Interviewers listen specifically for candidates who move down the spectrum when the requirement allows it.

Table 1.2: Six levels of agency, with the control point at each level

Level	Name	What the model controls	Representative system
0	Fixed pipeline	Nothing beyond text content	Template summariser
1	Router	Which of n fixed branches executes	Intent classifier plus handlers
2	Tool caller	Which tool runs and with what arguments, one step at a time	Retrieval assistant
3	Bounded loop	The sequence and count of steps, inside a hard cap	ReAct agent with a step ceiling
4	Planner-executor	A decomposition into subgoals, and replanning on failure	Deep research agent
5	Open-ended agent	Goal reformulation, tool acquisition, long-horizon persistence	Autonomous software maintainer

1.3 The Architecture Gap

A language model alone is missing five things that an acting system requires. Naming them individually is more useful than gesturing at “scaffolding,” because each gap maps to a component you will build.

- 1. No persistent state.** The model has no memory between calls. Any continuity you observe is a transcript you resent.
- 2. No side effects.** The model emits tokens. Something else must interpret those tokens as an intent to act and then act.
- 3. No grounding.** Parametric knowledge is frozen at training time and has no provenance. Anything the system asserts about current state must come from a retrieval or a tool call.
- 4. No verification.** The model cannot check its own output against the world. It can critique its own text, which is a weaker operation and correlates poorly with correctness on factual claims.
- 5. No termination guarantee.** Left to itself the model will happily continue producing plausible next steps. Termination is an engineering responsibility.

The reference architecture in Figure 1.1 closes those five gaps with six components. Every chapter in Parts I and II builds one of them in depth, and the capstone in Part IX assembles all six.

1.3.1 The Six Components and Their Contracts

State each component as a contract rather than as a box. Contracts are what allow you to swap implementations without redesigning the system, and interviewers listen for them.

Orchestrator Owns control flow, the step budget, the token budget, and the termination decision. Guarantees that every run terminates and emits a trace. Never calls an external system directly.

Reasoning model Given the current state and the tool schema, returns either a tool call or a final answer. Guarantees nothing about correctness, so treat every output as a proposal.

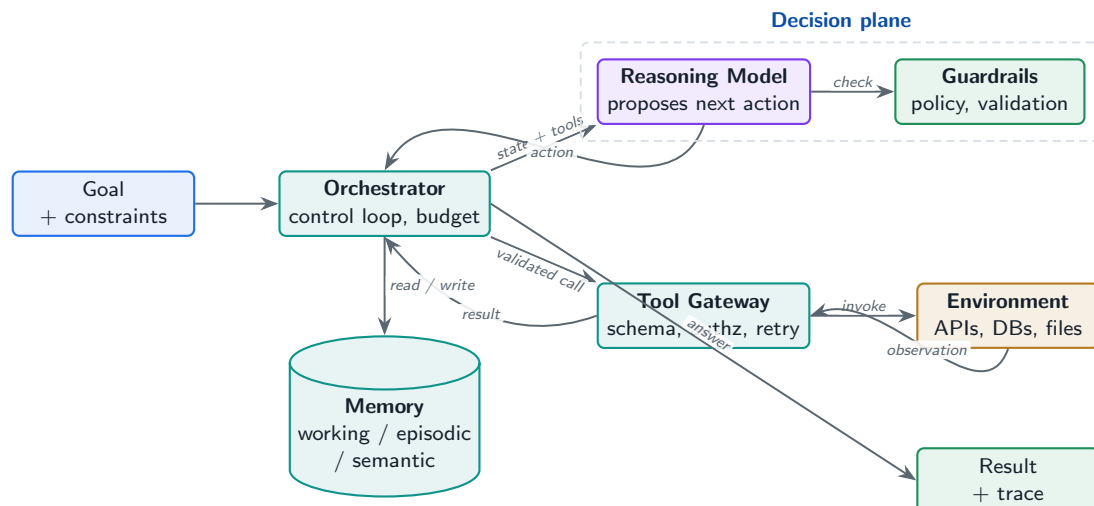


Figure 1.1: The six-component agent reference architecture. The orchestrator owns control flow, the model proposes, and the gateway is the only path to side effects.

Tool gateway Validates arguments against a schema, enforces authorisation for the calling principal, applies timeouts and retries, and returns a typed result or a typed error. Guarantees that the model cannot reach any system the policy did not permit.

Memory Provides working context for the current run, episodic recall of prior runs, and semantic recall of durable knowledge. Guarantees bounded read size so context construction stays deterministic.

Guardrails Evaluate inputs, proposed actions, and final outputs against policy. Guarantee that a violation halts or downgrades the run rather than passing through.

Observability Emits a span per step with token counts, cost, latency, tool identity, and outcome. Guarantees that any production run can be reconstructed after the fact.

★ The Gateway Is the Security Boundary

Notice that the model never touches the environment. It emits an intent, the orchestrator validates it, and the gateway executes it under a policy bound to the end user's identity. That single architectural choice converts prompt injection from a remote code execution vulnerability into a failed authorisation check. Chapter 16 develops the argument fully.

1.4 The Control Loop

The control loop is the heart of the orchestrator. Its structure is simple, and its correctness properties are where engineers get into trouble.

1.4.1 Termination Is Not Optional

A loop that can run forever will run forever eventually, and it will do so on the day a tool starts returning an ambiguous error. Bound the loop four independent ways, because each bound catches a different pathology.

1. **Step ceiling.** The blunt instrument. Catches oscillation between two states.

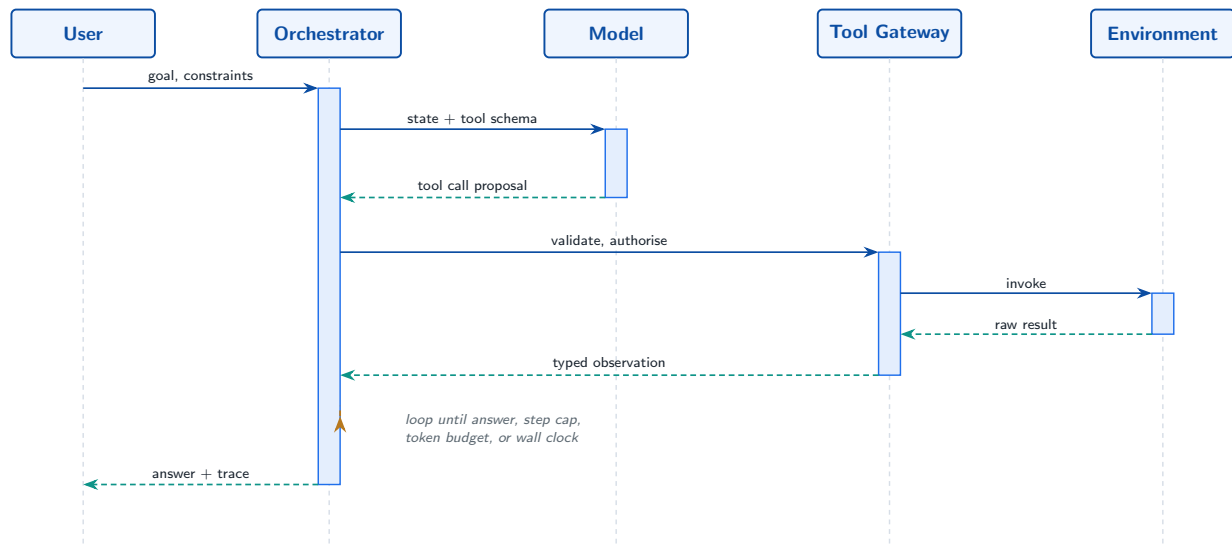


Figure 1.2: One iteration of the bounded control loop, drawn as a sequence over time. Every arrow that leaves the orchestrator is budget checked before it is sent.

2. **Token budget.** Catches the case where a single step returns an enormous observation and the model keeps re-reading it.
3. **Wall clock deadline.** Catches slow external dependencies that individually pass their time-outs.
4. **No-progress detector.** Catches the subtle case where the agent repeats the same tool call with the same arguments, which consumes budget without changing state.

```

1 Minimal production-shaped agent loop.
2
3 Design notes that matter more than the code:
4 * The model never executes anything. It returns a proposal, and the
5   orchestrator decides whether that proposal is allowed to run.
6 * Every termination path emits the same RunResult shape, so callers
7   never branch on did it work by inspecting exceptions.
8 * The budget ledger is updated before the call, not after, so a crash
9   mid-call still leaves an accurate spend record.
10
11
12 from __future__ import annotations
13
14 import hashlib
15 import time
16 from dataclasses import dataclass, field
17 from typing import Any, Protocol
18
19
20 class StopReason:
21     ANSWERED = answered
22     STEP_LIMIT = step_limit
23     TOKEN_LIMIT = token_limit
24     DEADLINE = deadline
25     NO_PROGRESS = no_progress
26     GUARDRAIL = guardrail_block
27
28

```

```

29 @dataclass
30 class RunBudget:
31     Every bound the orchestrator enforces, in one place.
32     max_steps: int = 12
33     max_tokens: int = 120_000
34     wall_clock_seconds: float = 90.0
35     repeat_threshold: int = 2          # identical calls tolerated before halt
36
37     tokens_used: int = 0
38     steps_used: int = 0
39     started_at: float = field(default_factory=time.monotonic)
40
41     def elapsed(self) -> float:
42         return time.monotonic() - self.started_at
43
44     def exhausted(self) -> str | None:
45         Return the StopReason that fired, or None while budget remains.
46         if self.steps_used >= self.max_steps:
47             return StopReason.STEP_LIMIT
48         if self.tokens_used >= self.max_tokens:
49             return StopReason.TOKEN_LIMIT
50         if self.elapsed() >= self.wall_clock_seconds:
51             return StopReason.DEADLINE
52         return None
53
54
55 @dataclass
56 class Step:
57     index: int
58     thought: str
59     tool_name: str | None
60     arguments: dict[str, Any]
61     observation: str | None
62     latency_ms: float
63     tokens: int
64
65
66 @dataclass
67 class RunResult:
68     answer: str | None
69     stop_reason: str
70     steps: list[Step]
71     tokens_used: int
72     elapsed_seconds: float
73
74     @property
75     def succeeded(self) -> bool:
76         return self.stop_reason == StopReason.ANSWERED
77
78
79 class ReasoningModel(Protocol):
80     def propose(self, state: AgentState) -> Proposal: ...
81
82
83 class ToolGateway(Protocol):
84     def schema(self) -> list[dict[str, Any]]: ...
85     def invoke(self, name: str, arguments: dict[str, Any]) -> str: ...
86
87

```

```

88 @dataclass
89 class Proposal:
90     What the model returns: either a final answer or a tool call.
91     final_answer: str | None
92     tool_name: str | None
93     arguments: dict[str, Any]
94     thought: str
95     tokens: int
96
97
98 @dataclass
99 class AgentState:
100     goal: str
101     scratchpad: list[Step] = field(default_factory=list)
102     tool_schema: list[dict[str, Any]] = field(default_factory=list)
103
104
105 class BoundedAgentLoop:
106     Orchestrator. Owns control flow, budget, and the termination decision.
107
108     def __init__(
109         self,
110         model: ReasoningModel,
111         gateway: ToolGateway,
112         budget: RunBudget | None = None,
113     ) -> None:
114         self._model = model
115         self._gateway = gateway
116         self._budget_template = budget or RunBudget()
117
118     def run(self, goal: str) -> RunResult:
119         budget = RunBudget(
120             max_steps=self._budget_template.max_steps,
121             max_tokens=self._budget_template.max_tokens,
122             wall_clock_seconds=self._budget_template.wall_clock_seconds,
123             repeat_threshold=self._budget_template.repeat_threshold,
124         )
125         state = AgentState(goal=goal, tool_schema=self._gateway.schema())
126         call_fingerprints: dict[str, int] = {}
127
128         while True:
129             fired = budget.exhausted()
130             if fired is not None:
131                 return self._finish(None, fired, state, budget)
132
133             started = time.perf_counter()
134             proposal = self._model.propose(state)
135             budget.tokens_used += proposal.tokens
136             budget.steps_used += 1
137
138             if proposal.final_answer is not None:
139                 self._record(state, budget, proposal, None, started)
140                 return self._finish(
141                     proposal.final_answer, StopReason.ANSWERED, state, budget
142                 )
143
144             # No-progress detection. An agent that reissues an identical call
145             # has stopped learning from its observations, so halt rather than
146             # burn the remaining budget discovering that fact slowly.

```

```

147     fingerprint = self._fingerprint(proposal)
148     call_fingerprints[fingerprint] = call_fingerprints.get(fingerprint, 0) + 1
149     if call_fingerprints[fingerprint] > budget.repeat_threshold:
150         self._record(state, budget, proposal, duplicate call, started)
151         return self._finish(None, StopReason.NO_PROGRESS, state, budget)
152
153     try:
154         observation = self._gateway.invoke(
155             proposal.tool_name or , proposal.arguments
156         )
157     except PermissionError as exc:
158         # A denied call is a guardrail event, not a crash. It halts the
159         # run so the denial is visible in the trace and in metrics.
160         self._record(state, budget, proposal, fdenied: {exc}, started)
161         return self._finish(None, StopReason.GUARDRAIL, state, budget)
162     except Exception as exc:
163         # Tool errors are observations. The model may recover from them,
164         # so feed the error back rather than aborting the run.
165         observation = ftool_error: {type(exc).__name__}: {exc}
166
167     self._record(state, budget, proposal, observation, started)
168
169     @staticmethod
170     def _fingerprint(proposal: Proposal) -> str:
171         payload = f{proposal.tool_name}|{sorted(proposal.arguments.items())}
172         return hashlib.sha256(payload.encode(utf-8)).hexdigest()[ :16]
173
174     @staticmethod
175     def _record(
176         state: AgentState,
177         budget: RunBudget,
178         proposal: Proposal,
179         observation: str | None,
180         started: float,
181     ) -> None:
182         state.scratchpad.append(
183             Step(
184                 index=budget.steps_used,
185                 thought=proposal.thought,
186                 tool_name=proposal.tool_name,
187                 arguments=proposal.arguments,
188                 observation=observation,
189                 latency_ms=(time.perf_counter() - started) * 1000.0,
190                 tokens=proposal.tokens,
191             )
192         )
193
194     @staticmethod
195     def _finish(
196         answer: str | None, reason: str, state: AgentState, budget: RunBudget
197     ) -> RunResult:
198         return RunResult(
199             answer=answer,
200             stop_reason=reason,
201             steps=state.scratchpad,
202             tokens_used=budget.tokens_used,
203             elapsed_seconds=budget.elapsed(),
204         )

```

Listing 1.1: A bounded agent control loop with four independent termination conditions and full trace emission

Three details in that listing carry disproportionate weight in a code review. First, a tool error becomes an observation rather than an exception, because recovering from a bad argument is exactly the behaviour you want. Second, an authorisation denial halts the run instead of becoming an observation, because feeding a denial back invites the model to search for a way around the policy. Third, every exit path constructs the same `RunResult`, so callers and dashboards never special-case.

Exercising the Loop From the Streamlit Lab

The companion lab exposes this loop with every bound editable, so you can watch each termination reason fire on demand.

1. Start the lab with `streamlit run chapter_01/app.py` and open the local URL it prints.
2. In the sidebar, set **Max steps** to 2 and submit a goal that needs three tool calls. Observe `stop_reason = step_limit` and note that a partial trace is still returned.
3. Set **Max steps** back to 12 and set **Wall clock** to 2 seconds against a tool with an injected delay. Observe `deadline`.
4. Enable **Force duplicate calls** in the debug panel. Observe `no_progress` after the third identical fingerprint.
5. Open the **Trace** tab and confirm every step shows tokens, latency, tool name, and observation. That table is the same data the production span exporter emits in Chapter 13.

1.5 A Working Taxonomy of Agents

Classical artificial intelligence divides agents into reflex, model-based, goal-based, utility-based, and learning agents (Russell & Norvig, 2021). That taxonomy remains useful for reasoning about what information the policy consumes. However, production teams need a second axis that describes how control flow is determined, because that axis predicts operational cost.

Workflow Versus Autonomous Agent

Forces. A fixed workflow gives you deterministic latency, a cost you can quote, and tests you can write. An autonomous loop handles inputs you did not enumerate.

Decision rule. Enumerate your traffic. If more than roughly eighty percent of requests follow a small number of paths that you can name, build the workflow and route the remainder to an agent. Interviewers reward this answer because it acknowledges that agency is a cost centre.

Measurement that settles it. Sample two hundred production requests, cluster them by the sequence of tools an expert would use, and count distinct sequences. Fewer than ten distinct sequences covering the bulk of traffic indicates a workflow. A long tail with no dominant sequence indicates an agent.

What you give up. Choosing the workflow means a class of requests fails or escalates. Choosing the agent means your **p99 latency** and your cost per request both become dis-

Table 1.3: Practical agent taxonomy, with the operational consequence of each type

Type	Control flow	Best fit	Operational cost
Reflex	Condition to action mapping, no state	Alerting, moderation, routing	Lowest, fully predictable
Workflow	Fixed graph, model fills the nodes	Known, repeatable processes	Low, latency is a sum
Router	Model picks one branch of n	Mixed intent traffic	Low, one extra call
Tool-calling	Model picks the next tool each turn	Question answering with actions	Moderate, variance grows
Planner-executor	Plan first, then execute and replan	Research, migrations, analysis	High, replanning is expensive
Multi-agent	Agents delegate to agents	Parallel or specialised subtasks	Highest, adds coordination

tributions rather than numbers, and every downstream service level objective inherits that variance.

1.6 When Not to Build an Agent

The most valuable judgment this book can give you is the judgment to say no. Four conditions each independently argue against an agentic design.

The task is deterministic. If a competent engineer can write the decision tree, write the decision tree. A model in that position adds latency, cost, and a nonzero error rate to a problem that had none.

Verification is impossible or expensive. Agents are viable in proportion to how cheaply you can check their work. Coding agents thrive because tests and compilers are cheap oracles. Financial advice agents struggle because the oracle is a human expert and a six-month outcome.

The error cost is asymmetric and unbounded. If one bad action can destroy data, move money irreversibly, or breach a regulation, then either the action needs a human approval gate or the system needs to be a recommender rather than an actor. Chapter 18 designs those gates properly.

Latency is hard bounded below the loop time. An agent that needs three model calls cannot serve a two hundred millisecond budget. Route those requests to a cached or classical path and reserve the agent for the tail.

The Agent-Shaped Hole

A team replaced a rules engine that handled refund eligibility with an agent. The rules engine ran in four milliseconds with perfect auditability. The agent ran in nine seconds, cost roughly two cents per decision, and agreed with the rules engine on about ninety-six percent of cases. The remaining four percent were not smarter, they were unexplainable. The team reverted, then used the agent for exactly the cases the rules engine marked as indeterminate,

which was under two percent of traffic. That is the correct shape.

1.7 The Cost Model You Will Be Asked to Defend

Every system design interview eventually reaches unit economics. Arrive with the arithmetic already assembled.

$$C_{\text{run}} = \sum_{i=1}^S (t_i^{\text{in}} \cdot p_{\text{in}} + t_i^{\text{out}} \cdot p_{\text{out}}) + \sum_j c_j^{\text{tool}}$$

The term that dominates is t_i^{in} , because most agent implementations resend the entire conversation plus every prior observation on every step. Consequently input tokens grow roughly quadratically in the number of steps, and a twelve step run can cost more than ten times a three step run rather than four times. Chapter 14 attacks this directly with caching, observation truncation, and summarisation checkpoints, and Chapter 7 attacks it structurally with memory design.

Work one example, because interviewers ask for numbers rather than for the formula. Assume six steps, an initial context of two thousand tokens, an average observation of eight hundred tokens appended per step, and four hundred output tokens per step. Input tokens across the run total roughly twenty-one thousand, output tokens total two thousand four hundred, and the run therefore costs an order of magnitude more than the naive estimate of six times a single call. Say that out loud in an interview and the conversation changes.

The Retry Storm That Tripled a Monthly Bill

System. A customer support agent at moderate scale, roughly forty thousand conversations per day, level three on the agency spectrum.

What happened. An upstream order lookup service began returning HTTP 200 with an empty body during a partial outage rather than an error status. The tool wrapper treated an empty body as a valid observation and returned the string `\`. The model, seeing no order, tried the same lookup with slightly reworded arguments, which produced a different fingerprint each time. The step ceiling of twenty was reached on roughly seventy percent of conversations for eleven hours.

Mechanism. Three defects compounded. The tool wrapper did not distinguish an empty result from a failed result. The no-progress detector fingerprinted exact arguments rather than semantic equivalence. The step ceiling was set high enough to be expensive and low enough to look like normal operation on the dashboard.

Resolution. The team introduced typed tool results with an explicit `EMPTY` versus `UNAVAILABLE` distinction, fingerprinted on the tool name plus a normalised argument hash rather than the raw arguments, lowered the default ceiling to eight with an explicit escalation path, and added a cost per conversation alert at three times the trailing median.

Transferable lesson. A degraded dependency that returns a successful status code is far more dangerous to an agent than one that fails cleanly, because failure is an observation the model handles and ambiguity is one it explores.

1.8 Where Agentic Systems Deliver in 2026

Four application classes have crossed from pilot into production at scale, and each one crossed for the same structural reason, namely cheap verification.

Software engineering. Coding agents plan, edit, run tests, and iterate. The test suite is the oracle, so a wrong action is detected within seconds at almost no cost. Chapter 21 covers the architecture.

Research and analysis. Deep research agents decompose a question, search in parallel, and synthesise with citations. Verification comes from source attribution, which a human can spot check quickly. Chapter 22 and the capstone build this.

Customer operations. Support agents resolve tickets end to end by calling internal systems. Verification comes from the transaction succeeding and from a downstream satisfaction signal.

Data and back office. Reconciliation, migration, and document processing agents handle high-volume tasks where a deterministic checker exists. Verification is a diff.

★ The Verification Heuristic

The commercial viability of an agentic application is governed less by model capability than by the cost of checking its work. Before you design anything, name the oracle. If you cannot name one, you are building a recommender with extra steps, and you should design it as one.

1.9 Chapter Summary

Key Takeaways

- An agent is a nondeterministic controller with credentials inside a distributed system. Goal-directedness, environment interaction, persistent state, and autonomy over control flow are the four testable properties, and the fourth is the discriminating one.
- Agency is a six-level spectrum. Design at the lowest level that meets the requirement, then justify each increment with a measurement.
- The reference architecture has six components: orchestrator, reasoning model, tool gateway, memory, guardrails, and observability. The model never touches the environment, which makes the gateway your security boundary.
- Bound every loop four independent ways: step ceiling, token budget, wall clock deadline, and a no-progress detector. Emit one result shape from every exit path.
- Treat tool errors as observations and authorisation denials as terminations. The distinction prevents an agent from probing your policy.
- Input tokens grow superlinearly with step count, so a six step run costs far more than six single calls. Bring that arithmetic to the interview.
- Cheap verification, not model capability, predicts which agentic applications reach production.

Lab Code for This Chapter

Run the reference implementation from [chapter_01/bounded_loop.py](#) in the companion repository at github.com/lamhotsiagian/agent-ai-lab. Run the loop against the mock tool suite, then use the sidebar controls to trigger each of the four termination reasons in turn and inspect the emitted trace for each one.

Designing Agent Systems

Subscribe → aiengineeringinsider.substack.com/subscribe

A repeatable method that turns a vague goal into a typed state machine, named component contracts, an explicit failure ladder, and numbers you can defend.

What This Chapter Delivers

This chapter gives you a seven-step design method you can execute at a whiteboard in forty minutes. You will learn to convert requirements into eight numbers before drawing anything, design a typed state schema that survives concurrent branches and checkpoint restores, write component contracts precise enough to swap implementations, choose among five core architecture patterns with a stated decision rule, and design a degradation ladder so that partial failure produces a partial answer rather than an exception. The chapter closes with the trade-off analysis interviewers probe hardest.

2.1 Overview: Design Precedes Framework

Most engineers approach an agent system by choosing a framework, then discovering their requirements through the shape of that framework's abstractions. The result works until it meets production traffic, at which point the team learns that the framework's default state model does not checkpoint, its default retry policy double-charges, and its observability hook fires after the token spend rather than before.

Design first. The framework is an implementation detail, and Chapter 4 shows that the significant frameworks are converging on the same primitives anyway. A candidate who opens a system design interview by naming a framework has signalled that they think in tools. A candidate who opens by asking what latency and cost the business will accept has signalled that they think in systems.

★ Design Order Is a Seniority Signal

The order in which you make decisions is itself evaluated. Requirements, then state, then contracts, then control flow, then framework. Interviewers who have run many loops can predict a candidate's level from the first three minutes purely by which of those five they reach for first.

2.2 The Seven-Step Design Method

The method below is deliberately mechanical. Under interview pressure, a mechanical process beats intuition, because intuition skips steps.

1. **Quantify the requirement.** Convert the prompt into the eight numbers in Table 2.1. Ask for them if they were not given.
2. **Name the oracle.** State how the system, or a human, will know that a run succeeded. If no oracle exists, redesign toward recommendation.
3. **Define the state.** Write the typed schema. State is the contract between every component, so it comes before the components.
4. **Draw the action space.** Enumerate the tools, their arguments, their side effects, and their blast radius.
5. **Choose the control pattern.** Pick from the five patterns in Section 2.4 using the stated decision rule.
6. **Design the failure ladder.** For each dependency, decide what a degraded response looks like before you decide what a healthy one looks like.
7. **Attach the measurement.** For every component you added, name the metric that would tell you it was the wrong choice.

2.2.1 Step One: The Eight Numbers

Requirements gathering in this field fails in a specific way. Engineers collect functional requirements (“it should answer questions about our documents”) and skip the quantitative envelope that actually determines the architecture.

Table 2.1: The eight numbers to establish before drawing a single box

Dimension	Question to ask	What the answer decides
Throughput	Concurrent runs at peak, and runs per day	Queue design, worker pool sizing, model quota
Latency	Acceptable p50 and p95 to first token and to completion	Synchronous versus asynchronous, streaming, step ceiling
Horizon	Typical and maximum steps per run	Checkpointing, memory strategy, cost envelope
Accuracy	Required task success rate, and how it is measured	Verification layer, reflection, human gate
Cost	Acceptable cost per successful run	Model tier routing, caching, early termination
Autonomy	Which actions may execute without approval	Guardrail placement, approval queue, audit depth
Data class	Sensitivity of the data the agent touches	Deployment locality, redaction, tenancy isolation
Recovery	Behaviour when a dependency is down	Degradation ladder, circuit breakers, fallbacks

✓ Say the Numbers Back

After collecting the eight numbers, restate them as a single sentence before you draw anything: “So we need two hundred concurrent runs, ninety-fifth percentile completion under

forty seconds, up to ten steps, ninety percent task success, under eight cents per successful run, writes require approval, customer data stays in region, and a search outage must still produce a cited partial answer.” That sentence is the design brief, and interviewers consistently mark it as a strong signal.

2.2.2 Step Three: State Before Components

State is the most under-designed part of most agent systems. Engineers treat it as a bag of variables that accumulates, then discover three problems in production: concurrent branches overwrite each other, checkpoint restores resume with stale tool results, and the context assembled from state exceeds the window on long runs.

Design state as a typed schema with an explicit merge rule per field. The merge rule is what makes concurrency safe.

```

1 Typed state for a research-and-analysis agent.
2
3 Three properties make this schema production ready:
4 * Every field declares how two concurrent branches merge. Without that
5   rule, a fan-out node silently loses one branch's writes.
6 * Budget lives in state, not in the orchestrator, so a checkpoint restore
7   resumes with the spend already incurred rather than a fresh allowance.
8 * Nothing in the schema holds a live connection or a callable, so the
9   whole object serialises for durable checkpointing.
10
11
12 from __future__ import annotations
13
14 import operator
15 from dataclasses import dataclass, field
16 from datetime import datetime, timezone
17 from enum import Enum
18 from typing import Annotated, Any, Literal, TypedDict
19
20
21 class RunPhase(str, Enum):
22     PLANNING = planning
23     EXECUTING = executing
24     VERIFYING = verifying
25     ESCALATED = escalated
26     COMPLETE = complete
27     FAILED = failed
28
29
30 @dataclass(frozen=True, slots=True)
31 class Evidence:
32     One retrieved or computed fact, with provenance attached.
33     claim: str
34     source_id: str
35     source_uri: str | None
36     confidence: float
37     retrieved_at: datetime
38
39     def as_citation(self) -> str:
40         return f"[{self.source_id}] {self.claim}

```

```

41
42
43 @dataclass(frozen=True, slots=True)
44 class SubGoal:
45     identifier: str
46     description: str
47     status: Literal[pending, running, done, blocked]
48     depends_on: tuple[str, ...] = ()
49
50
51 def merge_budget(left: dict[str, float], right: dict[str, float]) -> dict[str, float]:
52     Budget fields add across branches. Concurrent tool calls both spend.
53     merged = dict(left)
54     for key, value in right.items():
55         merged[key] = merged.get(key, 0.0) + value
56     return merged
57
58
59 def keep_latest_plan(left: list[SubGoal], right: list[SubGoal]) -> list[SubGoal]:
60     A replan replaces the plan wholesale. Appending would duplicate goals.
61     return right or left
62
63
64 class ResearchAgentState(TypedDict, total=False):
65     # --- immutable run identity -----
66     run_id: str
67     tenant_id: str
68     goal: str
69     submitted_at: datetime
70
71     # --- accumulating fields, safe to merge by concatenation -----
72     messages: Annotated[list[dict[str, Any]], operator.add]
73     evidence: Annotated[list[Evidence], operator.add]
74     tool_errors: Annotated[list[str], operator.add]
75
76     # --- replaced wholesale on each write -----
77     phase: RunPhase
78     plan: Annotated[list[SubGoal], keep_latest_plan]
79     draft_answer: str
80     verification_notes: str
81
82     # --- accounting, summed across concurrent branches -----
83     budget: Annotated[dict[str, float], merge_budget]
84
85     # --- loop control -----
86     step_count: int
87     replan_count: int
88     escalation_reason: str
89
90
91 def new_run_state(run_id: str, tenant_id: str, goal: str) -> ResearchAgentState:
92     Factory that guarantees every required field is present from step zero.
93     return ResearchAgentState(
94         run_id=run_id,
95         tenant_id=tenant_id,
96         goal=goal,
97         submitted_at=datetime.now(timezone.utc),
98         messages=[],
99         evidence=[],

```



```

100     tool_errors=[],
101     phase=RunPhase.PLANNING,
102     plan=[],
103     draft_answer=,
104     verification_notes=,
105     budget={input_tokens: 0.0, output_tokens: 0.0, usd: 0.0},
106     step_count=0,
107     replan_count=0,
108     escalation_reason=,
109 )

```

Listing 2.1: A typed agent state with per-field merge semantics, budget accounting, and checkpoint-safe serialisation

! The Fan-Out Data Loss Bug

The single most common concurrency defect in agent graphs looks like this. A node fans out to three parallel searches. Each branch writes `state["evidence"]` by assignment. The framework merges branches by taking the last writer, so two of the three searches vanish, and the agent confidently answers from one third of the evidence it paid for. Declaring the merge rule per field, as in the listing above, makes the defect impossible to express.

2.2.3 Step Four: The Action Space and Its Blast Radius

Enumerate every tool with four attributes, because those attributes determine where your guardrails go.

Table 2.2: Action space classification, which drives guardrail placement

Tool	Effect	Reversible	Blast radius	Control
<code>search_docs</code>	Read	Not applicable	Tenant scope	Rate limit
<code>run_sql_select</code>	Read	Not applicable	Row limit	Query allowlist, timeout
<code>send_email</code>	Write	No	External	Human approval
<code>create_ticket</code>	Write	Yes	One record	Idempotency key
<code>issue_refund</code>	Write	Costly	Money	Approval plus dollar cap
<code>execute_code</code>	Write	Depends	Sandbox only	Container, no network

The rule that falls out of this table is simple and worth stating explicitly in an interview. Read tools need rate limits and scope enforcement. Reversible write tools need idempotency keys. Irreversible write tools need a human in the loop or a hard numeric cap. Code execution needs isolation rather than validation, because validating arbitrary code is undecidable.

2.3 Component Contracts

Figure 2.1 shows the layered view that a design interview usually wants on the whiteboard. The value of drawing it in layers rather than as a free-form graph is that layers make the dependency direction explicit, and dependency direction is what interviewers probe when they ask “what happens if this box is down?”

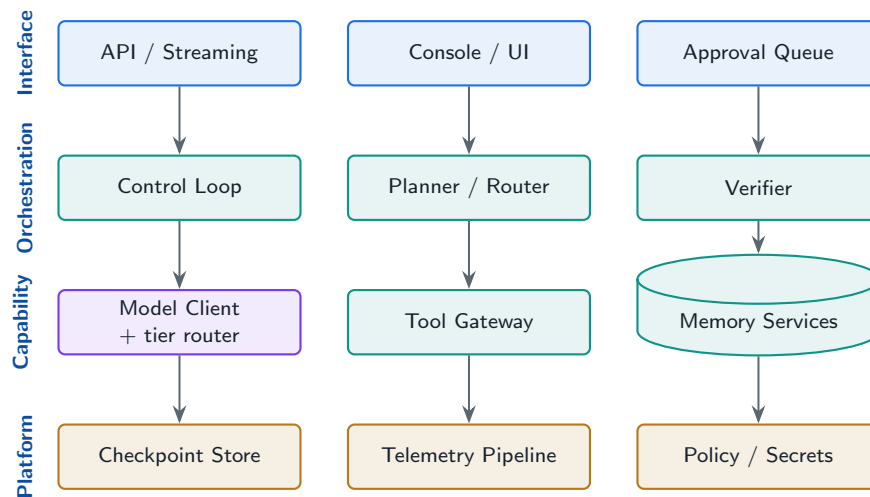


Figure 2.1: Layered agent architecture. Dependencies point downward only, so any layer can be tested with the layer below it replaced by a fake.

Write each contract as a signature plus an invariant. The invariant is the part that matters, and it is the part candidates usually omit.

```

1 from typing import Protocol, runtime_checkable
2
3
4 @runtime_checkable
5 class Planner(Protocol):
6     Decomposes a goal into an ordered plan.
7
8     Invariants:
9     * Returns at least one subgoal, or raises PlanningRefused with a reason.
10    * Every dependency identifier referenced by a subgoal exists in the plan,
11      so the executor never blocks on a goal that will not arrive.
12    * The returned plan is acyclic. The implementation checks this rather
13      than trusting the model.
14
15
16    def plan(self, goal: str, context: str) -> list[SubGoal]: ...
17
18
19 @runtime_checkable
20 class ToolGateway(Protocol):
21     The only path from the agent to any external effect.
22
23     Invariants:
24     * Rejects any tool not present in the caller principal's allowlist,
25       before the call is attempted and before any token is spent.
26     * Validates arguments against the declared JSON Schema and returns a
27       typed ValidationError rather than raising, so the model can correct.
  
```

```

28     * Applies a per-tool timeout and returns TimeoutResult rather than
29     blocking the orchestrator past its wall clock deadline.
30     * Returns a result object that distinguishes SUCCESS, EMPTY, DENIED,
31     UNAVAILABLE, and INVALID. Never returns a bare string.
32
33
34     def invoke(self, principal: str, name: str, arguments: dict) -> ToolResult: ...
35
36
37 @runtime_checkable
38 class MemoryService(Protocol):
39     Working, episodic, and semantic recall behind one interface.
40
41     Invariants:
42     * read() never returns more than max_tokens worth of content, so context
43     assembly is deterministic and the window cannot overflow.
44     * Every returned item carries a source identifier, so any claim the agent
45     makes can be traced to the memory that produced it.
46     * Writes are scoped to the tenant in the call, enforced in the store and
47     not only in the query.
48
49
50     def read(self, query: str, tenant_id: str, max_tokens: int) -> list[Evidence]: ...
51     def write(self, tenant_id: str, items: list[Evidence]) -> None: ...
52
53
54 @runtime_checkable
55 class Verifier(Protocol):
56     Checks a draft answer against the evidence that produced it.
57
58     Invariants:
59     * Returns a score in [0, 1] plus the specific unsupported claims, so the
60     orchestrator can decide between accept, revise, and escalate.
61     * Never modifies the draft. Verification and revision are separate steps
62     so that a failed verification is visible in the trace.
63
64
65     def verify(self, draft: str, evidence: list[Evidence]) -> VerificationResult: ...

```

Listing 2.2: Component contracts expressed as protocols with explicit invariants in the docstrings

2.4 Choosing the Control Pattern

Five patterns cover the overwhelming majority of production agent systems. Chapter 23 treats the full catalogue formally. Here is the decision rule you need at a whiteboard.

The plan-then-execute pattern deserves the sequence diagram in Figure 2.2, because its replanning edge is where most implementations are wrong. Replanning must consume the evidence gathered so far, and it must be bounded separately from the step ceiling, otherwise a plan that keeps failing produces an infinite replan cycle that never trips the step limit because each plan looks like progress.

Table 2.3: Control patterns with the condition that selects each one

Pattern	Select when	Cost profile	Main risk
Router	Intents are enumerable and disjoint	One extra small call	Misroute silently degrades quality
ReAct loop	Next step depends on the last observation, horizon is short	Grows with steps, superlinear in tokens	Oscillation, no-progress loops
Plan then execute	Subtasks are independent and parallelisable	Front-loaded, then parallel	Plan is made before evidence arrives
Reflection	A cheap critic materially improves output	Roughly doubles output tokens	Critic agrees with the generator
Hierarchical	Distinct specialities with different tools and data scopes	Highest, adds delegation overhead	Coordination cost exceeds benefit

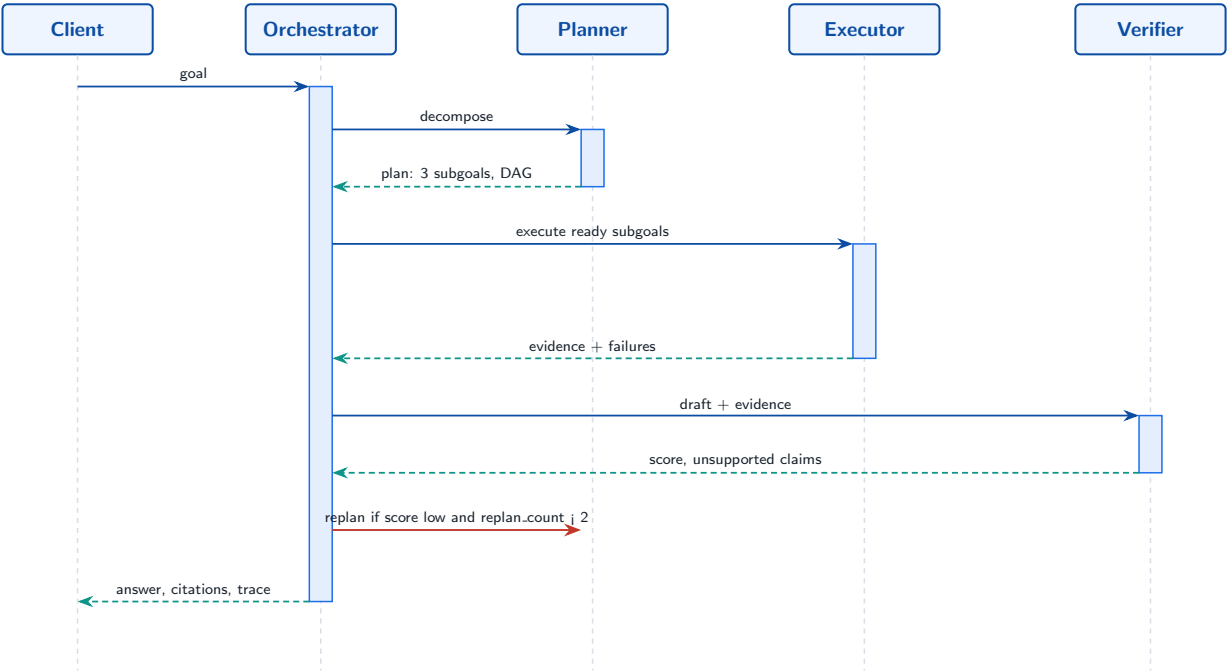


Figure 2.2: Plan, execute, verify, and bounded replan. The replan counter is separate from the step counter, which is what prevents a plausible-looking infinite cycle.

ReAct Versus Plan-Then-Execute

Forces. ReAct adapts to every observation but serialises everything, so latency is the sum of all steps. Plan-then-execute parallelises independent work but commits to a decomposition before any evidence exists.

Decision rule. If subgoal independence can be determined from the goal alone, plan first. If the second step depends on what the first step returned, use ReAct. Many real systems use both: plan at the top level, run ReAct inside each subgoal.

Numbers that decide it. For a research task with five independent sources, ReAct costs roughly five sequential model calls plus five tool latencies, so completion lands near thirty seconds. Plan-then-execute costs one planning call plus a parallel fan-out, so completion lands near ten seconds at a modest increase in total tokens. When the **p95 completion** requirement is under fifteen seconds, the choice makes itself.

What you give up. A plan built before evidence arrives will sometimes be wrong, which is why the replan edge exists and why it must be bounded.

2.5 Designing the Failure Ladder

Production agent systems fail partially far more often than they fail completely. A design that only distinguishes working from broken will surface partial failure as a confident wrong answer, which is the worst available outcome.

Define a degradation ladder per dependency, ordered from full capability to honest refusal, and make the current rung visible in the response.

Table 2.4: A degradation ladder for a research agent, from full capability to refusal

Rung	Condition	Behaviour	User-visible signal
0	All dependencies healthy	Full plan, parallel search, verification	Normal answer with citations
1	Reranker unavailable	Skip reranking, widen top- k	Answer, quality note in trace
2	Web search unavailable	Internal corpus only	“Answer based on internal sources only”
3	Primary model saturated	Route to smaller model, reduce step cap	Slower, shorter answer, banner
4	Vector store unavailable	Lexical search fallback	Explicit reduced-confidence label
5	No retrieval possible	Refuse with the specific reason	“Cannot answer, retrieval unavailable”

```

1 from dataclasses import dataclass
2 from enum import IntEnum
3
4
5 class Rung(IntEnum):
6     FULL = 0
7     NO_RERANK = 1
8     INTERNAL_ONLY = 2

```

```

9     SMALL_MODEL = 3
10     LEXICAL_ONLY = 4
11     REFUSE = 5
12
13
14 @dataclass(frozen=True)
15 class Capability:
16     Health snapshot, refreshed by the circuit breakers in Chapter 24.
17     reranker: bool
18     web_search: bool
19     primary_model: bool
20     vector_store: bool
21     lexical_store: bool
22
23
24 @dataclass(frozen=True)
25 class RunProfile:
26     rung: Rung
27     max_steps: int
28     top_k: int
29     model_tier: str
30     disclosure: str
31
32
33 class DegradationController:
34     Picks the best rung the current health state supports.
35
36     The controller is deliberately a pure function of the health snapshot.
37     Pure selection means the chosen rung is reproducible from a trace, which
38     is what makes a post-incident review possible.
39
40
41     def select(self, health: Capability) -> RunProfile:
42         if not health.vector_store and not health.lexical_store:
43             return RunProfile(
44                 Rung.REFUSE, 0, 0, none,
45                 Retrieval is unavailable, so this question cannot be answered
46                 from sources right now.,
47             )
48         if not health.vector_store:
49             return RunProfile(
50                 Rung.LEXICAL_ONLY, 4, 20, small,
51                 Semantic search is degraded. Results use keyword matching and
52                 may miss paraphrased passages.,
53             )
54         if not health.primary_model:
55             return RunProfile(
56                 Rung.SMALL_MODEL, 4, 8, small,
57                 Running on the standby model. Answers are shorter than usual.,
58             )
59         if not health.web_search:
60             return RunProfile(
61                 Rung.INTERNAL_ONLY, 8, 10, primary,
62                 Answer is based on internal sources only.,
63             )
64         if not health.reranker:
65             return RunProfile(
66                 Rung.NO_RERANK, 8, 18, primary,
67                 Ranking is degraded. Review the citations before acting.,

```

```

68     )
69     return RunProfile(Rung.FULL, 10, 8, primary, )

```

Listing 2.3: A degradation controller that selects the highest rung the current health state supports

✓ Make the Rung Observable

Emit the selected rung as a span attribute and as a dashboard dimension. Teams that do this discover that they spend a surprising fraction of their traffic on rung one or two without anyone noticing, which is exactly the kind of silent quality regression that erodes trust in an agent over months.

2.6 Testing an Agent Design Before You Build It

You can falsify a design on paper, which is much cheaper than falsifying it in production. Run these four checks.

The trace check. Write out one full trace by hand for the median request and one for the worst request you can imagine. Count steps, tokens, and seconds. If the worst case exceeds your budget by more than a factor of three, the design needs a bound you have not added yet.

The dependency check. For each external dependency, ask what the agent does when it returns slowly, when it returns an empty result, and when it returns a plausible but wrong result. The third case is the one that produces incidents.

The concurrency check. For each state field, ask what happens when two branches write it. If the answer is “the last one wins,” confirm that losing the other write is acceptable.

The restore check. Kill the process at every node boundary and ask whether resuming from the checkpoint would repeat a side effect. Every irreversible tool call needs an idempotency key for this reason.

The Checkpoint That Sent Every Email Twice

System. An outbound sales agent that researched a prospect, drafted an email, and sent it through a transactional provider, roughly eight thousand runs per day.

What happened. A rolling deploy restarted workers mid-run. The framework resumed each interrupted run from its last checkpoint, which had been written before the send node rather than after it. Roughly four hundred prospects received the same email twice within ninety seconds.

Mechanism. Checkpoints were written on node entry so that a crash inside a node could be retried. That choice is correct for read-only nodes and catastrophic for a node with an external side effect. Nobody had drawn the distinction because the state schema did not mark which nodes had effects.

Resolution. Three changes shipped together. Nodes were annotated as pure or effectful in the graph definition. Effectful nodes acquired a deterministic idempotency key derived from the run identifier plus the node name plus a hash of the arguments, which the provider deduplicated on. Checkpoints for effectful nodes moved to write-after-commit with the provider’s message identifier stored in state.

Transferable lesson. Checkpoint placement is a side-effect design decision, not a durability

setting. Ask where the checkpoint lands relative to every irreversible call before you choose a persistence backend.

Walking the Design in the Lab Console

The design console in the lab renders your state schema, action space, and degradation ladder as the three tables above, then simulates traces against mock dependencies so you can see the failure ladder fire.

1. Run `streamlit run chapter_02/app.py` and open the **Requirements** tab. Enter the eight numbers. The console computes an implied step ceiling and cost envelope and flags contradictions, for example a forty second **p95** with a twenty step ceiling.
2. Open **State Schema**. Paste the `ResearchAgentState` definition. The console lists every field without a declared merge rule, which is the fan-out data loss check.
3. Open **Failure Simulation**. Toggle each dependency off in turn and confirm the selected rung and disclosure text match Table 2.4.
4. Open **Trace Preview** and run the median and worst-case scenarios. Compare the reported step count, token count, and cost against the envelope you entered in step one.

2.7 Chapter Summary

Key Takeaways

- Design in a fixed order: requirements, oracle, state, action space, control pattern, failure ladder, measurement. The order is itself an evaluated signal.
- Convert the prompt into eight numbers before drawing a box. Restate them as one sentence to confirm the brief.
- Design state as a typed schema with a declared merge rule per field. Undeclared merges cause silent data loss on fan-out.
- Keep budget inside state so that a checkpoint restore resumes with the spend already incurred.
- Classify every tool by effect, reversibility, and blast radius. Read tools get limits, reversible writes get idempotency keys, irreversible writes get a human or a hard cap, and code execution gets isolation.
- Express component contracts as invariants, not just signatures. The invariant is what lets you swap implementations safely.
- Bound replanning separately from stepping, otherwise a repeatedly failing plan produces an infinite cycle that never trips the step limit.
- Design the degradation ladder before the happy path, and make the selected rung visible in the trace and in the response.

Lab Code for This Chapter

Run the reference implementation from `chapter_02/design.console.py` in the companion repository at github.com/lamhotsiagian/agent-ai-lab. Load the sample requirement sets,

then deliberately create a contradiction between latency and step ceiling and observe how the console reports it.

Reasoning Models and Test-Time Compute

Subscribe → aiengineeringinsider.substack.com/subscribe

Where an agent's planning quality actually comes from, what it costs per question, and how to route so that you buy reasoning only where it changes the answer.

What This Chapter Delivers

This chapter explains test-time compute as an engineering resource rather than as a model feature. You will learn how reasoning models differ mechanically from instruction-tuned models, why reinforcement learning on verifiable rewards produced the capability jump, how to think about the reasoning scaling curve and its saturation point, and how to build a router that spends reasoning budget only on requests where it moves the outcome. The chapter then implements the four reasoning topologies an agent can use, namely ReAct, ReWOO, self consistency, and reflection, with the measured conditions under which each one wins.

3.1 Overview: Compute Moved From Training to Inference

For most of the deep learning era, capability was purchased at training time. You spent more compute on pre-training, and the resulting model was better at everything, uniformly, forever. Inference cost was roughly fixed per token.

Reasoning models broke that separation. A model can now spend a variable amount of compute per question before producing its visible answer, and the additional compute buys accuracy on problems that decompose into verifiable steps. The consequence for system design is direct. Planning quality became a budget line rather than a model property, which means it is now something you allocate, meter, and route.

Test-Time Compute

Test-time compute is the additional inference-time computation a model spends on a single input before emitting its final response, whether through extended internal deliberation, sampling multiple candidate solutions, searching over intermediate states, or verifying and revising its own drafts. It is a resource allocated per request rather than a fixed property of the model.

The distinction matters for agents specifically because an agent makes many sequential decisions and the quality of each decision compounds. A planner that is right eighty-five percent of the time produces a five-step plan that is fully correct about forty-four percent of the time. Raising

per-step accuracy to ninety-five percent raises the full-plan rate to roughly seventy-seven percent. Consequently, spending reasoning budget on the planning step usually returns more than spending it on every step.

★ Reasoning Is a Compounding Investment at the Plan Step

Per-step accuracy compounds multiplicatively across a trajectory. Therefore the highest-return placement for expensive reasoning is the step whose output the most subsequent steps depend on, which is nearly always the plan. Interviewers who ask “where would you use a reasoning model in this design?” are testing for exactly this reasoning.

3.2 How Reasoning Models Differ Mechanically

Three mechanisms distinguish a reasoning model from an instruction-tuned model that has been prompted to think step by step.

Trained deliberation rather than prompted deliberation. Chain-of-thought prompting asks a model to produce intermediate tokens that happen to correlate with correct answers (Wei et al., 2022). A reasoning model is trained so that the intermediate tokens are the mechanism, using reinforcement learning against outcomes that can be checked automatically, principally mathematics, code, and formal logic (DeepSeek-AI, 2025). The difference shows up as robustness. Prompted deliberation degrades when the prompt changes. Trained deliberation persists.

Learned backtracking. Reasoning traces from these models contain explicit self-correction: the model pursues an approach, detects an inconsistency, and abandons it. That behaviour emerges from outcome-based reinforcement rather than from imitation of human-written solutions, which is why it appears even on problem types the training data did not contain.

A controllable budget. Production reasoning models expose an effort or thinking-budget parameter. Raising it lengthens deliberation and raises accuracy along a curve that saturates. That parameter is the most important knob in this chapter, because it is the one you route on.

3.2.1 The Scaling Curve and Where It Saturates

Accuracy against reasoning budget follows a concave curve. Early budget buys a great deal, and the marginal return falls steadily until additional budget buys nothing measurable and occasionally hurts, because a longer deliberation gives the model more opportunity to talk itself out of a correct early conclusion.

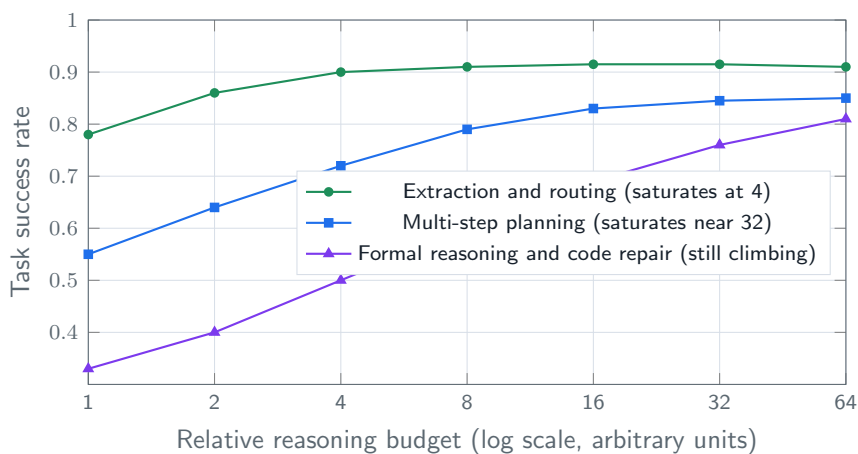
The practical consequence is that a single global effort setting is always wrong. Easy traffic pays for deliberation it does not need, and hard traffic is starved. Routing solves this, and routing is the deliverable of this chapter.

! Reasoning Tokens Are Billed and Often Invisible

Providers meter hidden reasoning tokens, and those tokens frequently exceed the visible output by a large multiple on hard problems. A team that meters only visible output will underestimate its bill substantially and will have no signal when a prompt change causes deliberation to double. Record reasoning token counts as a first-class metric from day one, and alert on the ratio of reasoning tokens to output tokens rather than on either alone.

Table 3.1: Instruction-tuned model versus reasoning model, from a system design perspective

Dimension	Instruction-tuned	Reasoning model
Latency	Roughly proportional to output length	Dominated by hidden deliberation, weakly predictable
Cost	Visible output tokens	Visible output plus reasoning tokens, often several times more
Best at	Extraction, formatting, routing, summarising, tool argument filling	Planning, multi-constraint decisions, debugging, mathematics, code
Worst at	Long multi-constraint planning	Simple formatting, where deliberation adds cost and sometimes drift
Streaming	Natural, token by token	First visible token is late, so design a progress affordance
Determinism	Moderate at low temperature	Lower, since deliberation paths vary
Agent placement	Executor steps, tool calls, summarisation	Planner, replanner, verifier, escalation triage

**Figure 3.1:** Accuracy against reasoning budget for three task classes. The saturation point differs by class, which is why a single global effort setting wastes money on easy traffic and starves hard traffic.

3.3 Routing Reasoning Budget

The router is a small, cheap classifier that decides how much deliberation a request receives. Build it as a cascade of increasingly expensive signals, so that most traffic is classified without a model call at all.

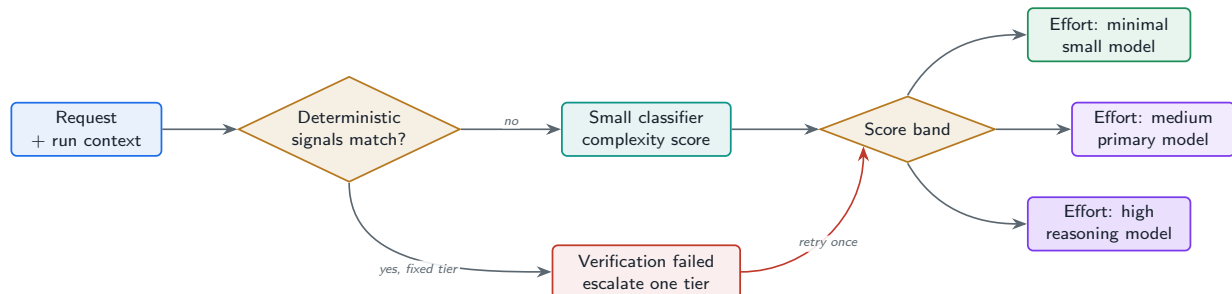


Figure 3.2: A three-stage reasoning router. Deterministic signals resolve most traffic, a small classifier resolves the remainder, and escalation on failed verification catches the rest.

```

1 Reasoning budget router.
2
3 The router answers one question: how much deliberation does this request
4 deserve? It is deliberately cheap, because a router that costs as much as
5 the call it is routing has no reason to exist.
6
7 Stage 1 deterministic signals ~0 ms, resolves the majority of traffic
8 Stage 2 small classifier call ~120 ms, resolves the ambiguous remainder
9 Stage 3 escalation on failure only after a verification miss
10
11
12 from __future__ import annotations
13
14 import re
15 from dataclasses import dataclass
16 from enum import Enum
17
18
19 class Effort(str, Enum):
20     MINIMAL = minimal
21     MEDIUM = medium
22     HIGH = high
23
24
25 @dataclass(frozen=True, slots=True)
26 class RoutingDecision:
27     effort: Effort
28     model_id: str
29     max_reasoning_tokens: int
30     reason: str # recorded on the span, so routing is auditable
31     stage: int # which stage decided, for router quality metrics
32
33
34 # Deterministic signals. Each one encodes a fact we know without asking a
35 # model, which is why stage one is nearly free.
36 _TRIVIAL_INTENTS = frozenset({
37     greeting, acknowledgement, cancel, status_check, list_options,
38 })
  
```